

Update 7: Ray Direct Transport, ROCm eRNIC, Near-SoL Collectives, and Where libscuffedrdma Sits in the libfabric / UCX / DOCA Landscape

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

September 2026

Contents

1	Summary	1
2	Ray Direct Transport	2
2.1	What RDT is	2
2.2	Semantics	2
2.3	The four optimizations, mapped	2
2.4	Why RDT has no dual QP pool, and why this work needs one	3
3	ROCm eRNIC	4
3.1	What it is	4
3.2	What it measures	4
3.3	Relevance	4
4	Every μs Matters (Near-SoL GPU Collectives)	4
4.1	What the paper shows	4
4.2	Relevance	5
5	The Transport Landscape: libfabric, UCX, DOCA	5
5.1	What the UCX papers add	5
5.2	What UCX says about itself	6
5.3	Where libscuffedrdma sits	6
6	RDMA Reliability Is an Industry Problem Too: GPFS	7
7	Consolidated Positioning	8
7.1	Implications for the thesis	8
8	Next Steps	8

1 Summary

Three recent developments sit close to the thesis: Anyscale’s Ray Direct Transport (RDT) [1, 2], AMD’s open-source ROCm eRNIC [7], and the “Every μ s Matters” paper on near speed-of-light GPU collectives [10]. This update reads each against libscuffedrdma, then places the middleware among the three transport stacks the industry builds on (libfabric, UCX, DOCA), prompted by internship work on GPFS, which uses libfabric underneath. A final section collects GPFS’s published RDMA defect record, which is the strongest available evidence that the reliability problems on this testbed are not artifacts of consumer hardware.

Two conclusions up front:

1. All three developments independently validate core design decisions already in the middleware: amortized memory registration, explicit routing instead of library heuristics, coalescing small transfers to the fabric’s efficient message size, and (in the SoL paper) treating decode-side small messages as the latency-critical class.
2. None of them classes traffic or schedules a contended link. Weight sync is homogeneous bulk; the SoL collectives are homogeneous small messages on NVLink; eRNIC is a device substrate. The mixed hot/cold problem on one commodity-Ethernet link is where this thesis sits.

Where the work stands. libscuffedrdma is a transport middleware built on libibverbs, under vLLM, for disaggregated prefill/decode over one 10GbE link (SoftRoCE today, RoCEv2 on ConnectX when available) between Cerberus and Chimera. In place: a dual QP pool with split polling, a size-plus-phase router, a PMP switching controller for bandwidth, the scuffedQuant 3-bit WHT+Lloyd-Max compressor for cold KV blocks, a pre-registered tensor-cache arena addressed by rkey offsets, and upstream UCX patches from the Update 4–5 work. The proposal’s broader agenda (joint training/inference scheduling, the full WFA classifier, KV migration, security) shaped this and can be picked back up if time allows; the main focus is the transport and its evaluation.

2 Ray Direct Transport

2.1 What RDT is

RDT lets a Ray actor return `torch.Tensors` that bypass Ray’s CPU-based plasma object store entirely. A producer method decorated with `@ray.method(tensor_transport="nixl")` keeps the tensor in GPU memory and stores only a reference; when the resulting `ObjectRef` is passed to a consumer actor, Ray triggers a direct GPU-to-GPU transfer through a third-party backend [3]. The work grew out of the GPU object store RFC [5].

Backend	Devices	Topology	Collective group?	Underneath
NIXL	CPU + GPU	point-to-point	no	UCX / GDS plugins
NCCL	NVIDIA GPU	collective	yes	ring/tree collectives
Gloo	CPU	collective	yes	TCP

Table 1: RDT transport backends [3]. NIXL is the flagship: point-to-point, no group setup, both CPU and GPU memory.

The detail most relevant to this thesis: NIXL’s network backend is UCX [6]. The entire RDT $\rightarrow$ NIXL $\rightarrow$ UCX stack ultimately posts to the same `ucp.put/ucp.get` paths that Updates 4 and 5 patched. Every UCX pathology documented in Update 5 (shared UDP source port on RoCE,

spinlock contention on shared `ucp_context`, the eager→RNDV latency cliff) sits underneath RDT too. RDT simply never encounters some of them because its benchmark fabrics (NVLink, InfiniBand) do not exercise the RoCE paths.

2.2 Semantics

Unlike plasma objects, RDT objects are *mutable references*: Ray does not copy the tensor until a transfer is requested, so in-place writes by the producer between `put` and transfer can leak partial updates to the consumer [3]. This is a deliberate trade: zero-copy on the producer side in exchange for weaker consistency. `libsuffledrdma` makes the same trade inside the tensor cache (KV blocks are registered in place, not staged), but the vLLM connector’s phase barrier, prefill completing before decode-side reads begin, closes the mutability window structurally. RDT pushes that responsibility onto application code.

2.3 The four optimizations, mapped

The RDT performance work [1] is a sequence of four optimizations over naive NIXL usage, benchmarked on 2 GB200 nodes with Multi-Node NVLink (900 GB/s point-to-point), transferring a 2 GB model as 1,000 tensor views to simulate tensor-parallel resharding.

1. **Register once, send views as offsets.** Naive NIXL registers each subtensor separately; RDT registers the base tensor once and describes each view as an offset into the registered region. Registration is the dominant per-transfer cost, so the speedup grows with view count. *Thesis mapping:* this is the tensor-cache design from Update 3, KV blocks inside a pre-registered arena, transfers as (offset, length) pairs against the arena’s rkey. Independent convergence from a production system, worth citing.
2. **Ahead-of-time registration.** `register_nixl_memory(param)` pre-registers memory before the first transfer, valuable when the same tensors ship every training step. *Thesis mapping:* equivalent to `RegisteredBuffer` in the dual QP pool, MRs registered once at pool creation and reused. Same amortization argument; a useful confirmation that per-transfer `ibv_reg_mr` is unacceptable in any serious deployment.
3. **Targeted receive (no staging buffer).** `set_target_for_ref(ref, views)` directs incoming data into the consumer’s existing tensors instead of a staging buffer; for a 20 GB weight sync this halves peak GPU memory (40 GB → 20 GB) [1]. *Thesis mapping:* `libsuffledrdma` writes KV blocks directly into the decode node’s paged KV arena via RDMA WRITE; there is no staging tier by construction. RDT needed an explicit API to recover what a purpose-built design gets for free.
4. **Memory pooling for small tensors.** `register_nixl_memory_pool(size, device)` pre-allocates a contiguous registered region and buckets small tensors into it, raising achieved bandwidth to 859 of the 900 GB/s hardware maximum [1]. *Thesis mapping:* the cold-QP coalescing strategy from Update 6, one layer up. RDT coalesces to fill NVLink packets; Update 6 coalesces compressed 50 KB KV blocks into 1 MB aligned writes. Same principle, different fabric.

Remaining overheads they report after all four optimizations: Python serialization of metadata objects over 100 KB, and $O(N)$ tensor-metadata bookkeeping for N views, milliseconds that survive even on a 900 GB/s fabric. On a 10GbE link those same milliseconds are noise. The binding constraint depends entirely on the fabric: RDT optimizes CPU-side software overhead because their network is effectively free; this thesis schedules the network because it is the scarce resource.

Result	Workload	Hardware	Baseline
7.5× end-to-end	2 GB / 1000 views	2× GB200 MNNVL	naive NIXL
18×, 3.5 s sync (SkyRL)	Qwen3-235B-A22B	4× 8×H100	NCCL broadcast
2.2× / 1.14×, 2.3 s (Miles)	GLM-4.5-Air	4× 8×H100	NCCL / Mooncake
up to 1000× transfer	large tensors	2× H100, 1 node	plasma object store

Table 2: RDT reported results [1, 2]. Every fabric is NVLink or InfiniBand-class; there are no commodity-Ethernet numbers.

2.4 Why RDT has no dual QP pool, and why this work needs one

Weight synchronization is a barrier workload: training pauses, weights ship, inference resumes. During the sync there is no competing latency-critical traffic; the entire link belongs to bulk transfer, and the only game is maximizing throughput and minimizing software overhead. RDT plays that game well.

Disaggregated prefill/decode is the opposite regime. Decode steps emit small, deadline-bound messages (block tables, sampling metadata, scheduler decisions) continuously, while prefill completions dump multi-megabyte KV slabs onto the same NIC. A single queue serializes them; that is the head-of-line blocking measured in Update 4. RDT on this workload would put everything through one NIXL path and inherit exactly the problem the dual QP pool solves. **RDT optimizes the transfer; libscuffedrdma schedules the link.**

The reverse list, for honesty: RDT ships in Ray, has a clean decorator API, handles arbitrary `torch.Tensor` resharding patterns, and is validated on frontier-scale hardware against Mooncake and NCCL. libscuffedrdma’s API surface (bootstrap + connector) is narrower and its benchmarks are two consumer nodes. RDT’s view-offset registration is also more general than the tensor cache’s fixed-arena scheme: it handles dynamically shaped views over any registered base tensor, where the arena assumes PagedAttention’s fixed block geometry. Ray lists a pluggable tensor-transport API as future work [4]; libscuffedrdma as a custom RDT backend, bringing traffic classing and dual QPs to Ray on commodity Ethernet, is a plausible post-thesis artifact.

3 ROCm eRNIC

3.1 What it is

ROCm/rocm-ernic [7] is a userspace *emulated* PCIe RDMA NIC for virtual machines: libvfiio-user transport, device logic adapted from QEMU’s PVRDMA, a guest kernel driver plus rdma-core provider exposing standard verbs (RC QPs; SEND, RDMA WRITE, RDMA READ; a DC-style DCT/DCI extension), and host-side backends of loopback, a TCP/IP mesh between eRNIC servers, or a real HCA via libibverbs. AMD positions it as a Soft-RoCE alternative for guests without RDMA hardware. MIT-licensed, actively committed, labeled an early-access preview.

3.2 What it measures

The repository’s own perf reports [8, 9] are the useful part. On the TCP mesh between two VMs on one EPYC host: RC RDMA WRITE at 4 KB averages $\sim 181 \mu\text{s}$; all verbs converge near 1.8–1.9 GB/s from 512 KB up; a bidirectional deadlock at high TX depth was fixed with per-direction processing flags, in-flight flow control of 4 WQEs, and WQE batching of 16. A GPU-initiated direct-verbs path

on an MI210 lands at ~ 1 ms per write, *flat from 64 B to 16 KB*; the report attributes this entirely to the doorbell and completion path, not data movement.

3.3 Relevance

eRNIC is not a substrate for this thesis (VM-only, ROCm GPU path, TCP-backed latency far above a hardware RoCE NIC). It matters in three narrower ways: a vendor-published software-RDMA latency floor to set beside my SoftRoCE measurements; an independent rediscovery of the split-per-direction, bounded-in-flight, batched-WQE discipline the dual QP pool uses; and a template for observability, since its Prometheus exporter reports per-instance RDMA-vs-TCP ratios and libscuffedrdma’s telemetry should expose the equivalent per-QP-class counters.

4 Every μ s Matters (Near-SoL GPU Collectives)

4.1 What the paper shows

Shen et al. [10] (NVIDIA and ETH Zürich; authors include Jeaugey, Hammond, and Hoefer) argue that GPU collectives are tuned for bandwidth while long-context, small-batch decode is latency-bound. They define a speed-of-light bound for a small AllReduce as the minimal data movement the hardware allows, $L_{\text{SoL}} = 2L_{\text{L2RTT}} + L_{\text{remote store}}$, measured at $1.404 \mu\text{s}$ on two GB200 ($0.306 \mu\text{s}$ L2 round trip, $0.792 \mu\text{s}$ remote store). Against that, NCCL ring on four GB200 sits at $11.0 \mu\text{s}$ for small messages. A single global memory barrier costs over $1 \mu\text{s}$; with two barriers in a $\sim 5 \mu\text{s}$ AllReduce, barriers alone are about 40% of the latency.

The design principles: barrier elimination; *data-as-signal* synchronization (LL-style flags embedded in the data, or sentinel-initialized buffers the receiver polls until real data overwrites them); double buffering with credits instead of barriers; direct use of symmetric memory and NVLink multicast. They ship these as an experimental low-latency API on NCCL’s device side (`ncclLLBuffer: send, recv, recvReduce, bcast`) plus a new two-shot LL128 atomic AllReduce. Results: small and medium messages within 7% of the SoL bound; $2.37 \mu\text{s}$ vs. $11.0 \mu\text{s}$ on four GB200 for the Llama-3.1-70B decode setting, an 8.7% inter-token-latency reduction; 7–13% ITL reduction across vLLM configurations; and their cost model puts each microsecond removed from AllReduce at about 0.9% of cost per million output tokens. The paper is explicit that its scope is a single scale-up (NVLink) domain; NCCL’s device API does reach inter-node over InfiniBand and RoCE through GPU-initiated networking, but multi-node versions of these collectives were not supported at the time of writing. They also note that most of the principles (LL, sentinel synchronization, double buffering) are not NVIDIA-specific.

4.2 Relevance

The thesis is inter-node commodity Ethernet, where the SoL bound is microseconds of wire plus NIC and the software stack adds far more. Two things transfer. First, “decode is latency-bound and small messages sit on every token’s critical path” is the premise of the hot traffic class, and this paper, with a measured ITL and cost-per-token argument, is a stronger citation for it than the NIM benchmarking numbers the proposal used. Second, their finding that explicit synchronization dominates small-message latency motivates one bounded experiment on the hot QPs: RDMA WRITE with the flag in the last bytes of the payload (LL style) or into a sentinel-initialized buffer, with the consumer polling, against CQ completion. The key difference in what is tested: they

remove *synchronization* latency inside a collective on a fabric where the link is not the bottleneck; the thesis removes *queueing* latency from bulk traffic sharing a link that is.

5 The Transport Landscape: libfabric, UCX, DOCA

Internship work on GPFS, which uses libfabric underneath, Brian Smith’s MUG’23 comparison of libfabric and UCX [14], and the two UCX papers [11, 12] clarified something the thesis has so far left implicit: which transport stack libscuffedrdma belongs to, and why.

	libfabric (OFI)	UCX	DOCA
Steward	OpenFabrics Alliance	UCF (NVIDIA-led)	NVIDIA
Design center	application-centric services	HPC primitives for MPI/SHMEM	BlueField DPU / ConnectX
Endpoint types	UD, RC, RDM, scalable EPs	UCP endpoints over UCT	DOCA RDMA, Verbs, GPUNetIO
Completion	CQs, counters, poll sets	callbacks	CQs, GPU-side polling
Protocol switch	per provider (eager/multi-pkt/RNDV)	eager / RNDV thresholds	driver / firmware
Portability	sockets provider runs anywhere	IB and RoCE primarily	NVIDIA hardware only
Typical users	MPI, GPFS, OPX/Omni-Path, EFA	MPI, NIXL, RDT, vLLM	DPU offload, GPUDirect

Table 3: The three stacks, from [14], the UCX FAQ [13], and the DOCA documentation [15]. Smith’s summary: UCX is efficient for building HPC primitives and uses callbacks; libfabric is application-centric, offers more endpoint options, and is thread-safe by default; “both provide very similar performance benefits.”

5.1 What the UCX papers add

The original UCX paper [11] lays out the three layers (UCS, UCT, UCP) and states that UCT “relies on low-level drivers provided by vendors such as InfiniBand Verbs, Cray’s uGNI, libfabrics,” so libfabric and UCX are not strictly alternatives: libfabric can sit under UCT. UCT defines short, bcopy, and zcopy operations; UCP adds transport selection, fragmentation, and multi-rail on top. Their prototype reached $0.89\mu\text{s}$ latency and 14M messages/s, close to the driver. Papadopoulos, Oden, and Balaji [12] then measured what UCP costs over UCT and verbs on InfiniBand and traced it to specific mechanisms: `RESOLVE_RKEY_RMA` re-resolving the remote key on every `ucp_put`; function-pointer dispatch from UCP into UCT that compilers cannot inline; the progress engine polling the receive CQ even for RMA-only traffic; and progress being called on the (x)UD wireup interface long after the RC connection is up. Skipping the unnecessary TX poll cut `ucp_ep_flush` instructions by 12%, skipping UD progress by 29.8%, and the latter improved small-message bandwidth and message rate by about 14%.

Three of those findings are the dual QP pool’s design, stated eight years earlier for a different reason: resolve keys once (the arena’s fixed rkey offsets), do not progress interfaces the operation does not use (split polling, so hot-QP polling never touches the cold CQ), and keep the software path between the application and the doorbell short. Their instruction-count methodology is also the right way to report the middleware’s own overhead in the evaluation chapter.

5.2 What UCX says about itself

The UCX FAQ [13] is clear that UCX “is not a user-level driver”: it relies on the RDMA, CUDA, and ROCm drivers underneath and adds software protocols where hardware lacks them. By default it “tries to use all available transports, and select best ones according to their performance capabilities and scale” (RC at small scale, DC or UD at large scale; the two best NICs for multi-rail), and it

picks eager-short, eager-bcopy, or rendezvous per message by size. Restricting that with UCX_TLS “can lead to unexpected and undefined behavior.” The only traffic-separation knobs it exposes are fabric-level: UCX_IB_SL and UCX_IB_TRAFFIC_CLASS set a service level or DSCP value for a context, which the NIC and switch must honor and which does not let an application route one message to one class and the next to another. That is the gap libscuffedrdma fills: it holds the QPs itself so the router, not the transport’s size heuristic, decides which queue a message enters.

5.3 Where libscuffedrdma sits

The middleware is built on libibverbs and keeps UCX out of its data path; the UCX pathologies from Updates 4 and 5, including the mmap interception leak observed in production vLLM deployments, are why the QPs are held directly. The upstream UCX patches stand on their own because NIXL, RDT, and vLLM’s existing connectors all route through UCX and inherit those fixes. Three observations follow from Smith’s deck.

1. **The eager/rendezvous cliff is not a UCX quirk.** OPX also selects eager, multi-packet eager, or rendezvous at runtime per message; libfabric providers do the same. In every stack the threshold is a provider decision the application does not see, which is the argument for routing traffic classes explicitly rather than tuning thresholds.
2. **libfabric’s endpoint model matches hot/cold directly.** libfabric exposes reliable-connected, reliable-unconnected (RDM), and unreliable datagram endpoints, capability bits (FI_RMA, FI_TAGGED, FI_FENCE), and lightweight counters as a completion mechanism. A hot class on an RDM endpoint with counters and a cold class on a connected endpoint with a CQ is a natural expression of the dual QP pool, and GPFS’s use of libfabric means the cold path’s 1 MB aligned writes could go through the same stack the file system already uses. A comparison worth running, not a rewrite: build the pool once on libfabric’s verbs provider over SoftRoCE and measure it against the libibverbs pool.
3. **DOCA is what most of the industry ships on, and it is out of reach here.** NVIDIA’s DOCA SDK [15] (DOCA RDMA, DOCA Verbs, GPUNetIO for GPU-initiated networking, Comch for DPU–GPU communication) is the production path on BlueField and ConnectX, and GPUDirect on this lab’s hardware already depends on the DOCA/MOFED host stack the proposal listed. What DOCA and its hardware provide, and what the thesis has to do in software, is traffic separation: DPU-side queues, hardware QoS, GPU-initiated doorbells. libscuffedrdma is a software stand-in for that hardware on a 10GbE NIC that has none of it, which is a clean way to state the contribution and its limit.

6 RDMA Reliability Is an Industry Problem Too: GPFS

The internship also surfaced how often production RDMA breaks, in a file system that has shipped RDMA for well over a decade. IBM Spectrum Scale (GPFS) publishes its RDMA defects as APARs and flash alerts, and they read like a checklist of what the middleware has to defend against.

- **Data integrity.** APAR IJ44629 [16]: a race between the Spectrum Scale RDMA layer and Mellanox adapters let clients read incorrect file data (5.1.3, fixed in 5.1.6.1; workaround was to disable RDMA). The cluster-wide alert behind APARs IJ44524/IJ44525 [17]: RDMA WRITES were not fully flushed from the HCA into client memory when PCIe atomics were unavailable, affecting 5.1.0.0–5.1.6.0 on Nvidia/Mellanox HCAs; IBM retired the **yes/no** values of `verbsRdmaWriteFlush` and made `rdma-read` (an RDMA READ as the flush) the default. A third alert [18]: an RDMA adapter failure on a non-ESS NSD server could turn file I/O into silent corruption instead of an I/O error.

- **Hangs and cluster loss.** A configuration where `verbsRdmasPerNode` was lower than `nsdMaxWorkerThreads` let the NSD server count an RDMA as complete before the data landed, producing crashes, hangs, and undetected corruption [19]. APAR IJ25511 [20]: with RDMA carrying all GPFS RPCs, a network error at the cluster manager left the newly elected manager blocked on a 10-second timeout and expelled every quorum node.
- **Fabric instability and silent fallback.** APAR IV61625 [21]: when a QP went to error state on the server, clients could not reconnect, the server kept the dead connection, and I/O fell back to TCP with the corresponding performance drop. On RoCE with `verbsRdmaCm`, non-link-local IPv6 addresses on the adapters caused hangs and long waiters [22].

Where GPFS is going. IBM Storage Scale 6.0.1 (June 2026) adds S3 over RDMA [23], built with NVIDIA’s `cuObject` libraries and tested on InfiniBand: “authentication, metadata, and control flows remain on HTTP, while bulk object payloads move over RDMA,” reaching about 90% of native POSIX throughput and roughly 300 GB/s GET / 155 GB/s PUT with GPU acceleration. That is a hot/cold split at the protocol level, small control traffic kept off the RDMA path entirely and bulk data on it, the same separation `libsuffledrdma` makes inside one link, one layer down. GPFS gets the separation by choosing different protocols per class on an InfiniBand fabric; the thesis has to make it inside verbs on 10GbE. GPFS’s own architecture already carries the parallel: the `verbsRdma/verbsRdmaSend` dual tier maps onto the hot/cold QP design, and its single-large-MR page pool mirrors the PagedAttention-style arena registration.

Relevance. Three of the defects map directly onto `libsuffledrdma`. The write-flush alert is the same completion-visibility question the data-as-signal experiment raises: an RDMA WRITE completion at the sender does not by itself guarantee the receiver’s memory is coherent, and GPFS’s answer (a trailing RDMA READ) is one of the options to measure against flag-in-payload polling on the hot QPs. The QP-error reconnection bug and the TCP fallback are exactly what the dual QP pool must handle when SoftRoCE or a ConnectX port flaps: a hot QP in error state has to be recreated without stalling the cold class, and a fallback must be visible in telemetry rather than silent. The `verbsRdmasPerNode` mismatch is a bounded-in-flight accounting bug, the same discipline eRNIC fixed with in-flight flow control and that the pool enforces per class. The point for the thesis: reliability and support on consumer hardware face the same failure modes the industry does on enterprise fabric, so the evaluation should report failure and recovery behavior next to latency.

7 Consolidated Positioning

7.1 Implications for the thesis

1. **Related work.** The chapter gets a two-axis taxonomy, layer (collective kernel / transport / orchestration / device) $\times$ traffic (homogeneous / mixed), annotated by stack (libfabric / UCX / DOCA). RDT, Mooncake, NIXL, and LMCACHE are homogeneous-bulk at the transport or orchestration layer on UCX; the SoL work is homogeneous-small at the kernel layer; eRNIC is a device-layer substrate; GPFS is a libfabric consumer with its own scheduling above the transport. SwiftRDMA remains the closest related work to the dual QP pool itself, with TransferEngine and Celeris the closest system-level comparisons. The mixed-traffic, commodity-Ethernet cell is where the thesis sits.
2. **Citable validation.** Registration amortization, explicit routing over library heuristics, and coalescing-to-fabric-size now have production precedent (RDT’s 7.5 $\times$); the decode-is-latency-

	RDT	ROCm eRNIC	Near-SoL collectives	libsuffledrdma
Layer	orchestration	emulated NIC	collective kernel	transport middleware
Stack	NIXL / UCX	verbs (emulated)	NCCL device API	libibverbs
Workload	RL weight sync	generic verbs	intra-node collectives	disagg. P/D KV
Traffic mix	homogeneous bulk	n/a	small, uniform	mixed hot + cold
Fabric	NVLink, IB	TCP / loopback / HCA	NVLink	10GbE SoftRoCE/RoCEv2
Registration	once + offsets	standard MR	symmetric memory	pre-registered arena
Sync model	CQ (via UCX)	CQ ring, doorbell	data-as-signal	CQ, split polling
Traffic classing	fabric-level	none	none	hot/cold dual QP pool
Bandwidth control	fabric-level	none	none	PMP switching controller
Compression	none	none	none	scuffedQuant

Table 4: The three systems against the thesis. Each shares mechanics with libsuffledrdma; classing and scheduling a contended link is left to the fabric in the others.

bound premise now has a measured ITL and cost-per-token citation [10]; the completion-visibility and QP-recovery requirements have a shipping file system’s defect record behind them.

3. **A benchmark worth running.** RDT + NIXL + UCX will run over SoftRoCE. The RDT weight-sync microbenchmark on the Cerberus/Chimera 10GbE link, where the 900 GB/s assumptions collapse, would show whether orchestration-layer RDMA degrades gracefully on commodity fabric.

8 Next Steps

1. Add RDT, eRNIC, the SoL paper, and the libfabric/UCX/DOCA framing to the related-work chapter; cite [10] for the decode-is-latency-bound premise.
2. Attempt `pip install nixl + Ray` on the cluster; verify NIXL selects the UCX backend over SoftRoCE. Then run RDT’s view-count sweep (1 to 10,000 views, fixed payload) on the 10GbE link, and repeat with a synthetic hot-traffic generator competing, to check whether a single-path transport shows the Update-4 HOL blocking.
3. Microbenchmark completion signaling on the hot QPs: CQ completion vs. flag-in-payload (LL style) vs. sentinel buffer vs. a trailing RDMA READ (the GPFS flush). Adopt only what moves p99. Report middleware overhead as instructions per operation, following [12].
4. Build the dual QP pool on libfabric’s verbs provider and compare against the libibverbs pool on identical payloads.
5. Add per-QP-class Prometheus counters, including QP error/reconnect and any TCP-fallback events, so telemetry is comparable to eRNIC’s exporter.

References

- [1] Anyscale, “RDT: Ray Direct Transport — Fast, Easy Weight Syncing for RL,” Anyscale Blog, 2026. <https://www.anyscale.com/blog/rdt-ray-direct-transport-fast-easy-weight-syncing-for-rl-reinforcement-learning>
- [2] Anyscale, “Ray Direct Transport: RDMA Support in Ray Core (Part 1),” Anyscale Blog, 2025. <https://www.anyscale.com/blog/ray-direct-transport-rdma-support-in-ray-core>
- [3] Ray Team, “Ray Direct Transport (RDT),” Ray Documentation. <https://docs.ray.io/en/master/ray-core/direct-transport.html>

- [4] Ray Team, “Implementing a Custom Tensor Transport (Advanced),” Ray Documentation. <https://docs.ray.io/en/latest/ray-core/direct-transport/custom-tensor-transport.html>
- [5] S. Wang et al., “[RFC] GPU Object Store Support in Ray Core,” ray-project/ray #51173, 2025. <https://github.com/ray-project/ray/issues/51173>
- [6] NVIDIA, “NIXL: NVIDIA Inference Xfer Library,” ai-dynamo/nixl documentation. <https://github.com/ai-dynamo/nixl/blob/main/docs/nixl.md>
- [7] AMD, “rocm-ernic: Userspace emulated RDMA NIC for virtual machines,” GitHub, 2026. <https://github.com/ROCm/rocm-ernic>
- [8] AMD, “ROCm eRNIC Performance Report,” docs/perf-report-2026-03-29.md and docs/performance.rst in [7].
- [9] AMD, “GPU Direct RDMA Test Report,” docs/gpu-rdma-test-report-2026-03-31.md in [7].
- [10] S. Shen, A. Korzh, A. Goel, T. Chen, J. Bachan, L. Schneider, P. Kousha, Z. He, S. Jeaugey, K. Iskra, N. Chandawala, J. R. Hammond, and T. Hoefler, “Every μ s Matters: Achieving Near Speed-of-Light Latency in GPU Collectives,” arXiv:2607.16100v1, July 2026. <https://arxiv.org/abs/2607.16100>
- [11] P. Shamis, M. G. Venkata, M. G. Lopez, M. B. Baker, O. Hernandez, Y. Itigin, M. Dubman, G. Shainer, R. L. Graham, L. Liss, Y. Shahar, S. Potluri, D. Rossetti, D. Becker, D. Poole, C. Lamb, S. Kumar, C. Stunkel, G. Bosilca, and A. Bouteiller, “UCX: An Open Source Framework for HPC Network APIs and Beyond,” *IEEE Hot Interconnects*, 2015.
- [12] N. Papadopoulou, L. Oden, and P. Balaji, “A Performance Study of UCX over InfiniBand,” *Proc. IEEE/ACM CCGrid*, 2017.
- [13] OpenUCX, “UCX Frequently Asked Questions,” docs/source/faq.md, openucx/ucx. <https://github.com/openucx/ucx/blob/master/docs/source/faq.md>
- [14] B. Smith, “Omni-Path and the Open Fabrics Interfaces,” Cornelis Networks, MVAPICH User Group (MUG) 2023.
- [15] NVIDIA, “NVIDIA DOCA SDK Documentation.” <https://docs.nvidia.com/doca/sdk/>
- [16] IBM, “APAR IJ44629: RDMA Scale client reads incorrect data from files,” IBM Support. <https://www.ibm.com/support/pages/apar/IJ44629>
- [17] IBM, “IBM Spectrum Scale Alert: Potential undetected data integrity issues for all clusters using RDMA (APARs IJ44524, IJ44525),” IBM Support. <https://www.ibm.com/support/pages/ibm-spectrum-scale-alert-potential-undetected-data-integrity-issues-all-clusters-using-rdma-0>
- [18] IBM, “IBM Spectrum Scale (GPFS): RDMA-enabled network adapter failure on the NSD server may result in file I/O error,” IBM Support. <https://www.ibm.com/support/pages/ibm-spectrum-scale-gpfs-rdma-enabled-network-adapter-failure-nsd-server-may-result-file-io-error>
- [19] IBM, “GPFS clusters enabled for RDMA use may experience server crashes, RDMA failures, hangs, or undetected data corruption,” IBM Support. <https://www.ibm.com/support/pages/gpfs-clusters-enabled-rdma-use-may-experience-server-crashes-rdma-failures-hangs-or-undetected-data-corruption>
- [20] IBM, “APAR IJ25511: All quorum nodes could be expelled,” IBM Support, 2020. <https://www.ibm.com/support/pages/apar/IJ25511>
- [21] IBM, “APAR IV61625: RDMA reconnection problem,” IBM Support. <https://www.ibm.com/support/pages/apar/IV61625>

- [22] IBM, “IBM Spectrum Scale 5.1.0.x resolved APARs.” https://public.dhe.ibm.com/storage/spectrumscale/spectrum_scale_apars_510x.html
- [23] T. Hoover, “IBM Storage Scale announces support for S3 over RDMA,” IBM Community Blog, June 16, 2026. <https://community.ibm.com/community/user/blogs/ted-hoover/2026/06/16/ibm-storage-scale-announces-support-for-s3-over-rd>
- [24] R. Qin et al., “Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving,” arXiv:2407.00079, 2024.
- [25] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *Proc. SOSP*, 2023.